A Neural-Network Quantum State for the Transverse-Field Ising Chain

Theory, Estimators, Architecture and Implementation
with a validated reference code

Working notes accompanying Carleo & Troyer, *Science* **355**, 602 (2017)
and Project 3, *A Neural-Network Quantum State for a Spin Chain*

August 21, 2026

What this document is. A self-contained treatment of the neural quantum state (NQS) method at the smallest scale where every claim can be checked exactly: the one-dimensional transverse-field Ising (TFI) chain at $N = 6$–12 spins. It is organised as a short textbook. Chapters 1–4 build the theory — Hilbert space, the variational principle, the Monte Carlo estimators, the restricted Boltzmann machine ansatz, and stochastic reconfiguration. Chapter 5 is the long one: it is a line-by-line account of network architecture and code, including every algebraic identity the implementation depends on, the autograd subtleties specific to PyTorch, and the numerical pitfalls that silently corrupt results. Chapter 6 covers the exact-diagonalisation and free-fermion benchmarks. Chapters 7–8 cover experimental protocol and the routes toward research-scale work.

Everything numerical here was run. The tables and the convergence figure in Chapters 4–7 come from the reference implementation reproduced in Chapter 5, executed against exact diagonalisation. Where a claim has *not* been checked numerically in this environment — specifically the PyTorch port in §5.10 — it is flagged explicitly.

Contents

1 The Many-Body Problem and Why Compression Is Possible **4**
1.1 The exponential wall . . . 4
1.2 The model system . . . 4
1.3 Boundary conditions and conventions . . . 5

2 The Variational Principle and Its Monte Carlo Estimators **6**
2.1 The bound . . . 6
2.2 The local energy . . . 6
2.3 The gradient is a covariance, not a backpropagated loss . . . 7
2.4 Statistical error and autocorrelation . . . 8

3 The Ansatz: Restricted Boltzmann Machines as Wave Functions **9**
3.1 Definition and the exact marginalisation . . . 9
3.2 The one dial: hidden density . . . 9
3.3 Structural remark: this is a Dense Associative Memory in disguise . . . 9
3.4 Log-derivatives . . . 10
3.5 Gauge redundancies . . . 10
3.6 Beyond the RBM . . . 11

4 Optimisation: Why Plain Gradient Descent Is Not Enough **12**
4.1 The problem with steepest descent . . . 12
4.2 The Fubini–Study metric and stochastic reconfiguration . . . 12
4.3 Numerical evidence . . . 13
4.4 What about Adam? . . . 14

5 Implementation and Network Architecture **15**
5.1 The repository landscape . . . 15
5.2 Design overview . . . 15
5.3 Working in log space . . . 16
5.4 Cached effective angles and the single-flip identity . . . 16
5.5 The Metropolis sampler . . . 17
5.6 The local energy . . . 18
5.7 Log-derivatives, batched . . . 19
5.8 Assembling and solving the SR equations . . . 19
5.9 The network class, complete . . . 19
5.10 The PyTorch port . . . 20
5.10.1 Complex parameters and the Wirtinger convention . . . 20
5.10.2 The surrogate objective . . . 20
5.11 Exact-summation mode: the debugging superpower . . . 21
5.12 Pitfalls . . . 22
5.13 Testing checklist . . . 22
5.14 Performance . . . 23

6 The Exact Benchmark **24**
6.1 Sparse construction and diagonalisation . . . 24
6.2 The free-fermion solution . . . 24

7 Protocol, Diagnostics and Deliverables **26**
7.1 Results the project asks for . . . 26
7.2 Diagnostics to log every run . . . 26
7.3 Suggested timeline, revised . . . 27

8 Extensions Toward Research **28**
8.1 Past exact diagonalisation . . . 28
8.2 Symmetry . . . 28
8.3 Where the method earns its keep: sign structure . . . 28
8.4 Architecture . . . 28
8.5 Dynamics . . . 28

A Notation **29**

B The One-Page Algorithm **30**

C Sources **31**

CHAPTER 1

The Many-Body Problem and Why Compression Is Possible

1.1 The exponential wall

A state of N spin-$\frac{1}{2}$ degrees of freedom lives in $\mathcal{H} = (\mathbb{C}^2)^{\otimes N}$, of dimension 2^N. Expanding in the product ("computational") basis of $\hat{\sigma}^z$ eigenstates,

$$|\Psi\rangle \;=\; \sum_{\mathbf{s}} \psi(\mathbf{s})\, |\mathbf{s}\rangle\,, \qquad \mathbf{s} = (s_1, \ldots, s_N), \quad s_i = \pm 1, \tag{1.1}$$

so the state is specified by the list $\{\psi(\mathbf{s})\}$ of 2^N complex numbers. At $N = 12$ that is 4096 numbers and fits in a cache line's worth of memory; at $N = 50$ it is 10^{15}; at $N = 300$ it exceeds the number of atoms in the observable universe. This is the exponential wall, and it is the reason the field exists.

The central observation. Physically relevant states are not typical points in $\mathcal{H}$. Ground states of local, gapped Hamiltonians occupy an exponentially small, highly structured corner of Hilbert space — they obey entanglement area laws, they have short correlation lengths, and their amplitudes are strongly correlated across configurations. A compressed parameterisation $\psi_{\mathcal{W}}(\mathbf{s})$ with $|\mathcal{W}| = \mathrm{poly}(N)$ can therefore be sufficient. Tensor networks exploit this via the entanglement structure; neural quantum states exploit it via generic function approximation.

1.2 The model system

The transverse-field Ising (TFI) chain is

$$\hat{H} \;=\; -J \sum_i \hat{\sigma}^z_i \hat{\sigma}^z_{i+1} \;-\; h \sum_i \hat{\sigma}^x_i. \tag{1.2}$$

The first term is classical and diagonal in our basis: it rewards alignment. The second is purely off-diagonal: $\hat{\sigma}^x_i$ flips spin i, and it is the sole source of quantum fluctuation. The competition between them produces a quantum phase transition at $|h/J| = 1$ in the thermodynamic limit: ferromagnetic order for $h < J$, paramagnetic disorder for $h > J$.

Two structural facts about (1.2) will be used repeatedly.

1. **Locality.** Each row of $\hat{H}$ in the product basis has at most $N + 1$ nonzero entries: one diagonal, and N off-diagonal ones reached by single spin flips. This is what makes the local-energy estimator of §2.2 cheap.

2. **Stoquasticity.** All off-diagonal matrix elements are $-h < 0$. By the Perron–Frobenius theorem the ground state can be chosen strictly positive in this basis, $\psi_0(\mathbf{s}) > 0$ for all $\mathbf{s}$. There is no sign problem, and a *real, positive* ansatz suffices.

Interpretive flag. Fact 2 is convenient and dangerous. It means the TFI chain will not exercise the complex-phase machinery of the ansatz at all: you can set $\mathrm{Im}\,\mathcal{W} = 0$ and lose nothing. Any conclusion you draw about "expressivity" from TFI alone does not transfer to a model with sign structure. This is precisely why the extension to a frustrated model (Chapter 8) is the one that turns the project into research.

1.3 Boundary conditions and conventions

Periodic boundary conditions (PBC, $i+1$ read mod N) give N bonds and preserve translation invariance; open boundary conditions (OBC) give $N-1$ bonds. At $N \leq 12$ the two differ by several percent in E_0, far above any variational error you will achieve, so *the choice must be fixed once and stated in every table*. Everything in this document uses PBC with $J = h = 1$ unless stated otherwise.

Note also that (1.2) and the more common convention $\hat{H}' = -J\sum \hat{\sigma}_i^x \hat{\sigma}_{i+1}^x - h\sum \hat{\sigma}_i^z$ are related by the global basis rotation $\hat{\sigma}^x \leftrightarrow \hat{\sigma}^z$ (a Hadamard on every site) and therefore have identical spectra. Literature values quoted for one apply to the other; only the identity of "the sampling basis" changes.

CHAPTER 2

The Variational Principle and Its Monte Carlo Estimators

2.1 The bound

Proposition 2.1 (Rayleigh–Ritz)**.** *For any $|\Psi\rangle \neq 0$ in the domain of $\hat{H}$,*

$$E[\Psi] \;=\; \frac{\langle\Psi|\,\hat{H}\,|\Psi\rangle}{\langle\Psi|\Psi\rangle} \;\geq\; E_0, \tag{2.1}$$

with equality iff $|\Psi\rangle$ is a ground state.

Proof. Expand in the eigenbasis, $|\Psi\rangle = \sum_n c_n |n\rangle$ with $\hat{H}|n\rangle = E_n|n\rangle$. Then $E[\Psi] - E_0 = \sum_n |c_n|^2 (E_n - E_0) / \sum_n |c_n|^2 \geq 0$, since every $E_n \geq E_0$. Equality forces $c_n = 0$ whenever $E_n > E_0$. □

This is the entire logical content of the method: restrict $|\Psi\rangle$ to a parametric family $|\Psi_{\mathcal{W}}\rangle$, minimise $E[\mathcal{W}]$ over $\mathcal{W}$, and the result is a certified upper bound on E_0. Nothing guarantees the bound is *tight*; that is an empirical question about the expressivity of the family, and it is why benchmarking against exact diagonalisation is not optional.

2.2 The local energy

Written out in the product basis, (2.1) is a ratio of sums over 2^N terms, which is exactly what we cannot afford. The standard manipulation converts it into an expectation over a probability distribution we *can* sample.

Insert two resolutions of the identity:

$$E[\mathcal{W}] = \frac{\sum_{\mathbf{s},\mathbf{s}'} \psi^*(\mathbf{s})\,\langle\mathbf{s}|\,\hat{H}\,|\mathbf{s}'\rangle\,\psi(\mathbf{s}')}{\sum_{\mathbf{s}} |\psi(\mathbf{s})|^2} = \frac{\sum_{\mathbf{s}} |\psi(\mathbf{s})|^2 \sum_{\mathbf{s}'} \langle\mathbf{s}|\,\hat{H}\,|\mathbf{s}'\rangle\,\dfrac{\psi(\mathbf{s}')}{\psi(\mathbf{s})}}{\sum_{\mathbf{s}} |\psi(\mathbf{s})|^2}, \tag{2.2}$$

where the second form follows by multiplying and dividing by $\psi(\mathbf{s})$. Defining the *Born distribution* and the *local energy*,

$$p(\mathbf{s}) \;=\; \frac{|\psi(\mathbf{s})|^2}{\sum_{\mathbf{s}''} |\psi(\mathbf{s}'')|^2}, \qquad E_{\mathrm{loc}}(\mathbf{s}) \;=\; \sum_{\mathbf{s}'} \langle\mathbf{s}|\,\hat{H}\,|\mathbf{s}'\rangle\,\frac{\psi(\mathbf{s}')}{\psi(\mathbf{s})}, \tag{2.3}$$

we obtain

$$\boxed{E[\mathcal{W}] \;=\; \langle E_{\mathrm{loc}}\rangle_p \;=\; \mathbb{E}_{\mathbf{s}\sim p}\big[E_{\mathrm{loc}}(\mathbf{s})\big].} \tag{2.4}$$

Two properties deserve emphasis.

- **Locality makes it cheap.** The sum in (2.3) runs only over the $\mathbf{s}'$ connected to $\mathbf{s}$ by $\hat{H}$. For the TFI chain this is N single-flip terms plus the diagonal, so E_{loc} costs $O(N)$ wave-function *ratios* — never absolute amplitudes, and never a normalisation.

- **The normalisation cancels.** Both p and E_{loc} are ratio quantities. The ansatz never needs to be normalised, which is what makes an unnormalisable 2^N-component object tractable at all.

For the TFI chain specifically, with $\mathbf{s}^{(k)}$ denoting $\mathbf{s}$ with spin k flipped,

$$\boxed{E_{\mathrm{loc}}(\mathbf{s}) \;=\; -J\sum_{i} s_i s_{i+1} \;-\; h\sum_{k=1}^{N}\frac{\psi(\mathbf{s}^{(k)})}{\psi(\mathbf{s})}.} \tag{2.5}$$

Remark 2.2 (Zero-variance principle)**.** *If $|\Psi_{\mathcal{W}}\rangle$ is exactly an eigenstate, then $\hat{H}\,|\Psi\rangle = E\,|\Psi\rangle$ implies $E_{\mathrm{loc}}(\mathbf{s}) = E$ for* every $\mathbf{s}$*, and* $\mathrm{Var}[E_{\mathrm{loc}}] = 0$*. The variance is therefore a quality metric independent of knowing E_0: it is the diagnostic to plot alongside the energy, and it is the one that still works when you scale past exact diagonalisation.*

2.3 The gradient is a covariance, not a backpropagated loss

Define the *log-derivatives* (variational derivatives, "O-operators")

$$O_k(\mathbf{s}) \;=\; \frac{\partial \log \psi_{\mathcal{W}}(\mathbf{s})}{\partial p_k}, \tag{2.6}$$

where p_k ranges over the parameters. Differentiating (2.4) requires care because p itself depends on $\mathcal{W}$. For real parameters the result is

$$\boxed{\frac{\partial E}{\partial p_k} = 2\,\mathrm{Re}\Big[\big\langle \big(O_k - \langle O_k\rangle\big)^{*}\big(E_{\mathrm{loc}} - \langle E_{\mathrm{loc}}\rangle\big)\big\rangle\Big].} \tag{2.7}$$

Sketch. Write $E = \langle E_{\mathrm{loc}}\rangle$ with the $\mathcal{W}$-dependence in both E_{loc} and p. Differentiating the Born weight gives $\partial_k \log p(\mathbf{s}) = 2\,\mathrm{Re}\; O_k(\mathbf{s}) - \langle 2\,\mathrm{Re}\; O_k\rangle$, contributing $\langle (2\,\mathrm{Re}\, O_k - \langle 2\,\mathrm{Re}\, O_k\rangle) E_{\mathrm{loc}}\rangle$. Differentiating E_{loc} itself and using hermiticity of $\hat{H}$ collapses the two contributions into the single centred form above. The centring by $\langle O_k\rangle$ is exact (it subtracts a term proportional to $\langle E_{\mathrm{loc}}\rangle \cdot 0$) and is a pure variance reduction — but a large one, and it should never be omitted. □

For *complex* holomorphic parameters it is cleaner to define the *force*

$$F_k \;=\; \langle O_k^{*}\, E_{\mathrm{loc}}\rangle - \langle O_k^{*}\rangle\,\langle E_{\mathrm{loc}}\rangle \;=\; \big\langle \big(O_k - \langle O_k\rangle\big)^{*}\big(E_{\mathrm{loc}} - \langle E_{\mathrm{loc}}\rangle\big)\big\rangle\,, \tag{2.8}$$

which is the Wirtinger derivative $\partial E/\partial p_k^{*}$; steepest descent on the real scalar E is then $\delta p_k = -\eta F_k$, and (2.7) is the specialisation $2\,\mathrm{Re}\, F_k$ to real parameters.

Conventions differ by factors of two and by conjugation across the literature and across codebases. Do not trust any of them, including this document. Verify your gradient once against a central finite difference on a tiny system, $\partial_k E \approx [E(p_k + \epsilon) - E(p_k - \epsilon)]/2\epsilon$ evaluated by full summation over all 2^N configurations. This is a five-line test and it catches the single most common class of bug in this project (§5.13).

Why this is not a normal deep-learning loss. Autograd cannot differentiate through the Monte Carlo sampler: the samples are drawn from a distribution that depends on the parameters, and the sampling operation is not differentiable. Equation (2.7) is a score-function (REINFORCE-type) estimator, and it must be assembled by hand or injected through a surrogate objective. Backpropagating through E_{loc} as though it were a supervised loss produces a gradient that is *wrong*, not merely noisy, because it omits the $\partial_k \log p$ term.

The implementation of this point is §5.10.2.

2.4 Statistical error and autocorrelation

With N_s samples from a Markov chain of integrated autocorrelation time τ,

$$\mathrm{err}[E] \;\approx\; \sqrt{\frac{\mathrm{Var}[E_{\mathrm{loc}}]}{N_s}\,\big(1+2\tau\big)}. \tag{2.9}$$

The naïve error bar (with $\tau = 0$) is optimistic, often by an order of magnitude, and reporting it is a standard way to convince yourself you have converged when you have not. Estimate τ by binning: split the chain into blocks of length b, compute the variance of the block means, and increase b until that variance stops growing. Alternatively, use many short independent chains — the approach taken in the reference code, where 64 chains are advanced in lockstep and the sample buffer is refreshed every iteration.

CHAPTER 3

The Ansatz: Restricted Boltzmann Machines as Wave Functions

3.1 Definition and the exact marginalisation

The Carleo–Troyer ansatz is a restricted Boltzmann machine over N visible spins $s_i = \pm 1$ and M hidden spins $h_j = \pm 1$, with *complex* parameters $\mathcal{W} = \{a_i, b_j, W_{ij}\}$:

$$\psi_M(\mathbf{s}; \mathcal{W}) \;=\; \sum_{\{h_j = \pm 1\}} \exp\Big(\sum_i a_i s_i + \sum_j b_j h_j + \sum_{ij} W_{ij}\, s_i h_j \Big). \tag{3.1}$$

Because the graph is bipartite — no visible–visible or hidden–hidden couplings — the hidden sum factorises site by site and can be performed *exactly*:

$$\begin{aligned} \psi_M(\mathbf{s}; \mathcal{W}) &= e^{\sum_i a_i s_i} \prod_{j=1}^{M} \sum_{h_j = \pm 1} e^{h_j (b_j + \sum_i W_{ij} s_i)} \\ &= e^{\sum_i a_i s_i} \prod_{j=1}^{M} 2\cosh\big(\theta_j(\mathbf{s})\big), \qquad \theta_j(\mathbf{s}) \;=\; b_j + \sum_i W_{ij} s_i. \end{aligned} \tag{3.2}$$

The quantities θ_j are the *effective angles*; caching them is the single most important performance decision in the implementation (§5.4).

In log form, which is the only form the code should ever manipulate,

$$\boxed{\log \psi(\mathbf{s}) \;=\; \mathbf{a}\cdot\mathbf{s} \;+\; \sum_{j=1}^{M} \log\cosh \theta_j(\mathbf{s}) \;+\; M \log 2.} \tag{3.3}$$

The constant $M \log 2$ is irrelevant — every quantity in the method is a ratio — but keeping it costs nothing and makes comparisons against a direct evaluation of (3.1) exact.

3.2 The one dial: hidden density

The only architectural hyperparameter is the *hidden density*

$$\alpha \;=\; M/N, \tag{3.4}$$

which gives a parameter count $|\mathcal{W}| = N + M + NM = N(1 + \alpha + \alpha N)$ — quadratic in N at fixed α, against 2^N for the exact state. The family is nested in α (padding with zero-weight hidden units reproduces a smaller network), so $E[\alpha]$ is monotonically non-increasing in α up to optimisation error, and α plays the role that bond dimension plays in matrix product states.

3.3 Structural remark: this is a Dense Associative Memory in disguise

Equation (3.3) has the form

$$\log \psi(\mathbf{s}) \;=\; \mathbf{a}\cdot\mathbf{s} + \sum_{\mu=1}^{M} F\big(\langle \xi^\mu, \mathbf{s}\rangle + b_\mu\big), \qquad F(x) = \log\cosh x, \tag{3.5}$$

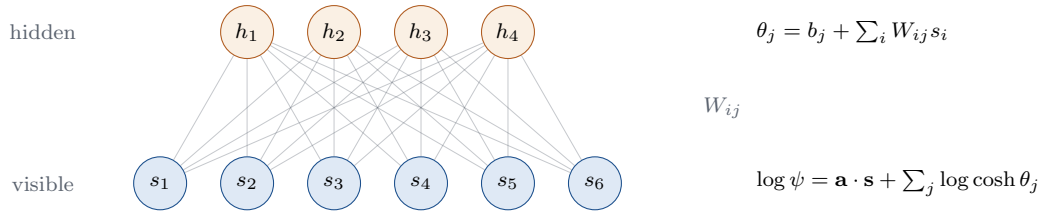


Figure 3.1: The RBM architecture with $N = 6$, $M = 4$ ($\alpha = 2/3$). The bipartite structure is what permits the exact hidden-unit trace (3.2): there is no approximation anywhere in going from (3.1) to (3.3).

with the columns $\xi_i^\mu = W_{i\mu}$ playing the role of stored patterns. This is exactly the Dense Associative Memory / modern-Hopfield energy with separation function F: an analyse ($\mathbf{s} \mapsto$ overlaps), sharpen (F), synthesise pipeline, with the weight matrix a rank-M operator factoring through pattern space. The correspondence is structural, not incidental — RBMs and Hopfield networks are the same object viewed from two directions.

Where the analogy breaks, and it matters. In an associative memory, the energy $E(\mathbf{s})$ *is* the dynamical object: states relax down its gradient. Here, $\log \psi(\mathbf{s})$ is not an energy at all — it is an amplitude. The physical energy is a *separate* quadratic functional $\langle\Psi| \hat{H} |\Psi\rangle / \langle\Psi|\Psi\rangle$ of that amplitude, and the optimisation descends in $\mathcal{W}$, not in $\mathbf{s}$. Three further disanalogies: (i) the parameters are complex, so $\log \psi$ has a phase with no statistical-mechanics counterpart; (ii) $|\psi|^2$, not ψ, is the Born probability, so the "Boltzmann weight" of the RBM is the amplitude's square modulus; (iii) there are no stored patterns to retrieve — the ξ^μ are free parameters fit to a Hamiltonian, not memories imprinted by a learning rule. Carrying Hopfield intuition across without these caveats will mislead.

3.4 Log-derivatives

Differentiating (3.3) gives the O_k needed by (2.7) in closed form:

$$O_{a_i}(\mathbf{s}) = s_i, \qquad O_{b_j}(\mathbf{s}) = \tanh \theta_j(\mathbf{s}), \qquad O_{W_{ij}}(\mathbf{s}) = s_i \tanh \theta_j(\mathbf{s}). \tag{3.6}$$

Note $O_{W_{ij}} = O_{a_i} O_{b_j}$: the full $(N+M+NM)$-vector is an outer product plus two marginals, and can be formed with one tanh evaluation and one broadcast. There is no reason to use automatic differentiation for an RBM; the closed forms are faster and exact. (For deeper architectures, §3.6, autograd becomes the right tool.)

3.5 Gauge redundancies

Three exact invariances of (3.2) are worth knowing, because they make the geometry of $\mathcal{W}$-space singular and are part of why the optimiser of Chapter 4 needs regularisation:

1. **Hidden-unit sign flip:** $(b_j, W_{\cdot j}) \to (-b_j, -W_{\cdot j})$ leaves $\cosh \theta_j$ unchanged. 2^M-fold degeneracy.
2. **Hidden-unit permutation:** relabelling j leaves ψ unchanged. $M!$-fold.
3. **Periodicity of the complex log:** $\theta_j \to \theta_j + 2\pi i$ is invisible; $\theta_j \to \theta_j + i\pi$ flips the sign of cosh and can be absorbed into a global phase.

Consequently the quantum geometric tensor S of §4.2 always has exact zero modes and *cannot* be inverted directly. This is a structural fact, not a numerical accident.

3.6 Beyond the RBM

The RBM is one choice among many, and swapping it is the natural expressivity study.

Ansatz	**Structure**	**Trade-off**
RBM	$\log\psi = \mathbf{a}\cdot\mathbf{s} + \sum_j \log\cosh\theta_j$	Closed-form derivatives; shallow; permutation-blind unless symmetrised.
Symmetric RBM	Weights shared across translations	Imposes momentum-zero sector; far fewer parameters, usually more accurate at fixed α.
Feedforward net	$\log\psi = f_{\mathcal{W}}(\mathbf{s})$, any depth	Arbitrary expressivity; needs autograd; harder to condition.
CNN	Convolutional f	Manifest translation covariance; the standard choice in 2D.
Autoregressive (RNN / PixelCNN / transformer)	$\psi(\mathbf{s}) = \prod_i \psi(s_i \mid s_{<i})$	*Exact, uncorrelated* sampling — no Markov chain, no τ; normalisation is automatic. Costs an $O(N)$ sequential pass per sample.

The autoregressive row is the one to note: it removes an entire class of bugs and diagnostics (burn-in, autocorrelation, ergodicity) at the price of a more expensive forward pass. If the project's sampling never quite behaves, that is the escape hatch.

CHAPTER 4

Optimisation: Why Plain Gradient Descent Is Not Enough

4.1 The problem with steepest descent

Gradient descent $\mathcal{W} \to \mathcal{W} - \eta\, \nabla_{\mathcal{W}} E$ is steepest descent with respect to the *Euclidean* metric on parameter space, $\|\delta\mathcal{W}\|^2 = \sum_k |\delta p_k|^2$. But parameter space is a coordinate chart on a manifold of quantum states, and the Euclidean metric has no physical meaning there: reparameterising the network changes the trajectory while leaving the state manifold untouched. Combined with the exact gauge degeneracies of the previous section, this produces an extremely ill-conditioned optimisation. Empirically it does not fail dramatically; it converges to a plateau one to three orders of magnitude above the achievable energy and sits there, which is far more insidious.

4.2 The Fubini–Study metric and stochastic reconfiguration

The physically meaningful distance between two normalised states is the Fubini–Study distance. Expanding it to second order in $\delta\mathcal{W}$ gives the metric

$$\boxed{S_{kk'} \;=\; \langle O_k^* O_{k'} \rangle - \langle O_k^* \rangle \, \langle O_{k'} \rangle} \tag{4.1}$$

— the *quantum geometric tensor*, a covariance matrix of log-derivatives. (For real parameters and real ψ this reduces to the classical Fisher information of $p(\mathbf{s})$, up to a factor of 4; the natural-gradient literature and the variational Monte Carlo literature are describing the same object.)

Steepest descent in this metric — equivalently, the projection of an imaginary-time step $|\Psi\rangle \to (1 - \delta\tau \hat{H})\,|\Psi\rangle$ back onto the variational manifold — gives *stochastic reconfiguration* (SR):

$$\boxed{\sum_{k'} S_{kk'}\, \delta p_{k'} \;=\; -\eta\, F_k, \qquad F_k = \langle O_k^* E_{\mathrm{loc}} \rangle - \langle O_k^* \rangle \, \langle E_{\mathrm{loc}} \rangle \,.} \tag{4.2}$$

This is exactly natural gradient descent; in the machine-learning idiom, S^{-1} is a preconditioner that undoes the arbitrary curvature introduced by the parameterisation.

Regularisation

S is singular by construction (§3.5) and is in any case estimated from N_s samples, so for $N_s < |\mathcal{W}|$ its sample estimate has rank at most $N_s - 1$. Two standard fixes:

$$\text{diagonal shift:} \quad S \to S + \lambda\, \mathrm{diag}(S), \quad \lambda : 10^{-2} \to 10^{-4} \text{ on a decay schedule;} \tag{4.3}$$

$$\text{explicit shift:} \quad S \to S + \epsilon \mathbb{1}, \quad \epsilon \sim 10^{-3}. \tag{4.4}$$

The reference code uses the first with $\lambda_t = \max(\lambda_0\, 0.99^t,\, 10^{-5})$.

The kernel trick (minSR)

Forming S explicitly costs $O(N_s|\mathcal{W}|^2)$ time and $O(|\mathcal{W}|^2)$ memory, which becomes the bottleneck the moment the network is large. Write the centred, $\sqrt{N_s}$-normalised derivative matrix $\bar{O} \in$

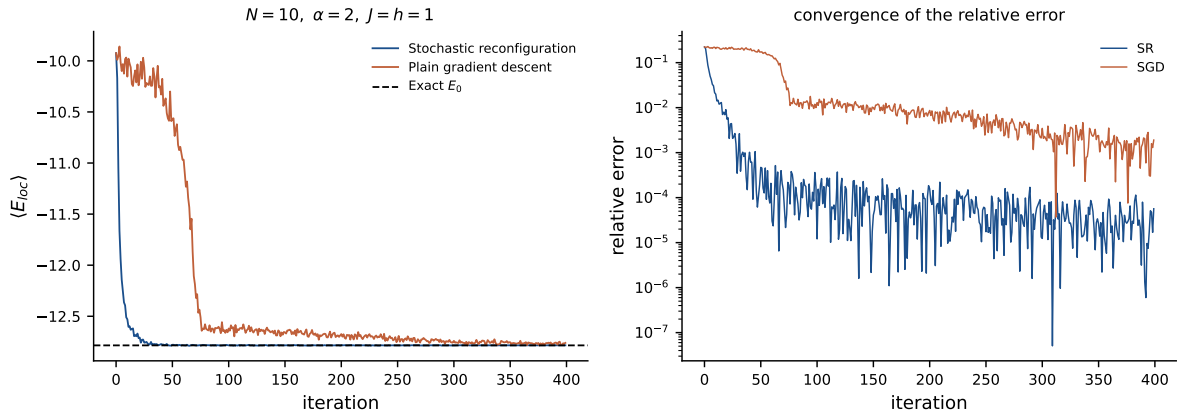


Figure 4.1: Left: energy per iteration for stochastic reconfiguration and for plain gradient descent, against the exact $E_0 = -12.78490644$. Right: the same data as relative error on a log scale. SR falls by three further orders of magnitude; SGD plateaus. Produced by the reference implementation of Chapter 5.

$\mathbb{C}^{N_s \times |\mathcal{W}|}$ and centred local energies $\varepsilon \in \mathbb{C}^{N_s}$; then $S = \bar{O}^\dagger \bar{O}$ and $F = \bar{O}^\dagger \varepsilon$, and the Woodbury identity gives

$$\delta\mathcal{W} \;=\; -\eta\, \bar{O}^\dagger \big(\bar{O}\bar{O}^\dagger + \lambda \mathbb{1}\big)^{-1} \varepsilon, \tag{4.5}$$

which requires only the $N_s \times N_s$ Gram matrix. When $N_s \ll |\mathcal{W}|$ — the regime of every deep architecture — this is the only sensible formulation.

4.3 Numerical evidence

Figure 4.1 and Table 4.1 compare SR against plain gradient descent on the identical ansatz, sampler, and sample budget. Both runs use $N = 10$, $\alpha = 2$ (230 complex parameters), 64 chains, 16 sweeps per iteration, 400 iterations. The final energies are evaluated by *exact summation* over all 2^{10} configurations of the converged network, so the numbers below carry no Monte Carlo error at all.

Table 4.1: Optimiser comparison at $N = 10$, $\alpha = 2$, PBC, $J = h = 1$. Final energies by exact summation over the full basis.

Optimiser	**Learning rate**	**Final E**	**Relative error**
Exact diagonalisation	—	-12.78490644	—
Stochastic reconfiguration	0.05	-12.78486410	3.3×10^{-6}
Plain gradient descent	0.02	-12.76735829	1.4×10^{-3}

Conclusion. The factor of $\sim$400 in accuracy between the two rows of Table 4.1 is the difference between a project brief that says "$\vartheta \to \vartheta - \eta \nabla_\vartheta E$" and the method that produced the Carleo–Troyer results. Plain descent is adequate for a first working loop and for debugging; it is not adequate for the deliverable table of ML-versus-exact energies. Implement SR.

4.4 What about Adam?

Adam and other adaptive first-order methods are diagonal preconditioners: they rescale each coordinate by a running estimate of its gradient magnitude. They help, and they are much cheaper than SR, but they cannot capture the *off-diagonal* correlations between parameters that S encodes, which is precisely where the ill-conditioning lives. A reasonable practical ladder: Adam for the first few tens of iterations to escape the initialisation, then SR for convergence. Do not use SR with an aggressive learning rate from a random start — the sample estimate of S is poor when the state is delocalised, and the step can be wild.

CHAPTER 5

Implementation and Network Architecture

This chapter is the working core of the document. It covers what the code actually looks like, every algebraic identity the implementation leans on, the two or three places where a correct-looking program silently computes the wrong thing, and a complete reference implementation that has been run against exact diagonalisation.

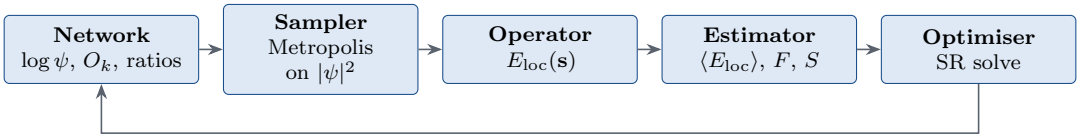
5.1 The repository landscape

Repository	Stack	What it is
netket/netket	JAX / Flax	The reference implementation, from Carleo's group at EPFL. Production quality: samplers, operators, SR/minSR drivers, symmetry groups, fermions. Use it as ground truth.
bartolsthoorn/NQS-numpy	NumPy	∼400-line notebook reimplementation of the paper. The best thing to read first.
zeldredge/py-nqs	NumPy	Python NQS in the Carleo–Troyer style, including time evolution.
athete/neural-quantum-states	NumPy	Compact single-purpose implementation.
remmyzen/nqs-tensorflow	TensorFlow	GPU port, based on NetKet; also implements transfer-learning protocols across system size.

On "the code of the paper". The 2017 Science paper's own program was a small standalone C++ code; what has a maintained public home is its descendant, NetKet. I was not able to verify a canonical public repository for the original standalone code itself, so treat NetKet as the lineage rather than the artefact. Note also that NetKet 3 is built on JAX, *not* PyTorch — the project brief recommends both, which are two different stacks. The sane division of labour is: write your own loop in PyTorch (that is the pedagogy), and use NetKet purely as an independent cross-check on the numbers.

5.2 Design overview

Five components, and the dependency structure is a straight line:



Keep these genuinely separate. The single most valuable structural decision is that **the network exposes ratios, not amplitudes**: an interface `log_psi_ratio(s, flips)` returning $\log[\psi(\mathbf{s}')/\psi(\mathbf{s})]$ is used by both the sampler and the local energy, and is where all the caching lives. If you write the sampler against absolute amplitudes you will pay $O(NM)$ per proposal instead of $O(M)$ and you will overflow.

Data layout

Object	Shape / dtype	Notes
`s`	(n_{chains}, N) float	Spins as ± 1 floats, *not* bits or booleans — every formula is written in ± 1 and converting on the fly is a persistent source of sign errors.
`theta`	(n_{chains}, M) complex	Cached effective angles, kept in sync with `s` at all times.
`a, b`	$(N,)$, $(M,)$ complex	Visible and hidden biases.
`W`	(N, M) complex	Note the orientation: $\theta = b + \mathbf{s}W$.
`eloc`	$(n_{\text{samples}},)$ complex	Complex even for a stoquastic Hamiltonian; take `.real` only at the very end.
`O`	$(n_{\text{samples}}, \lvert\mathcal{W}\rvert)$ complex	The dominant memory cost: $N_s \times N(1+\alpha+\alpha N)$.

5.3 Working in log space

ψ itself must never be materialised. With $M = \alpha N$ hidden units each contributing a factor $2\cosh\theta_j$, amplitudes span hundreds of orders of magnitude and overflow `float64` at modest N. Everything is done on $\log\psi$, and every physical quantity is a difference of logs exponentiated once, at the end, after the difference is known to be small.

The one function that needs care is $\log\cosh$ of a complex argument. The naive `log(cosh(x))` overflows for $\operatorname{Re} x \gtrsim 710$. Since cosh is even, fold to the half-plane $\operatorname{Re} u \geq 0$ by $u = x\,\operatorname{sgn}(\operatorname{Re} x)$ and use

$$\log\cosh(x) \;=\; \log\cosh(u) \;=\; u + \log\bigl(1 + e^{-2u}\bigr) - \log 2, \qquad \bigl|e^{-2u}\bigr| \leq 1. \tag{5.1}$$

Every term is now bounded. This works verbatim for complex u (`numpy.log1p` and `torch.log1p` both accept complex input), and it is exact — not an asymptotic approximation.

```python
def logcosh(x):
    """cosh is even, so fold to Re(u) ≥ 0 where |exp(-2u)| ≤ 1."""
    sgn = np.where(np.real(x) ≥ 0, 1.0, -1.0)
    u = sgn * x
    return u + np.log1p(np.exp(-2.0 * u)) - np.log(2.0)
```

Listing 5.1: Stable $\log\cosh$, valid for real and complex argument.

5.4 Cached effective angles and the single-flip identity

This is the trick that makes the whole method affordable, and it is the part most often reimplemented badly.

Proposition 5.1 (Single-flip update)**.** *Let* $\mathbf{s}^{(k)}$ *denote* $\mathbf{s}$ *with* $s_k \to -s_k$*. Then*

$$\theta_j(\mathbf{s}^{(k)}) \;=\; \theta_j(\mathbf{s}) \;-\; 2s_k W_{kj}, \tag{5.2}$$

and consequently

$$\boxed{\log \frac{\psi(\mathbf{s}^{(k)})}{\psi(\mathbf{s})} \;=\; -2s_k a_k \;+\; \sum_{j=1}^{M} \Big[\log\cosh\left(\theta_j - 2s_k W_{kj}\right) - \log\cosh\theta_j \Big].} \tag{5.3}$$

Proof. $\theta_j(\mathbf{s}) = b_j + \sum_i W_{ij} s_i$ is affine in $\mathbf{s}$; flipping s_k changes the kth term from $W_{kj}s_k$ to $-W_{kj}s_k$, a change of $-2s_k W_{kj}$. The visible-bias term changes by $a_k(-s_k) - a_k s_k = -2s_k a_k$. Substituting into (3.3) and subtracting gives (5.3). □

> **Why this matters.** Recomputing θ from scratch costs $O(NM)$. Using (5.2) against a cached θ costs $O(M)$. Over a sweep of N proposals per configuration, per iteration, per chain, this is the difference between a project that runs in seconds and one that runs in minutes — and at the sizes you will want for the extensions, between feasible and not. In Carleo's original code these caches are called "lookup tables".

The invariant to maintain is: *`theta` always equals `rbm.theta(s)` for the current `s`*. Update it on every accepted move and never otherwise. A cheap assertion comparing the cached and recomputed values, enabled under a debug flag, is worth writing once.

5.5 The Metropolis sampler

The chain is over configurations $\mathbf{s}$ with stationary distribution $p(\mathbf{s}) \propto |\psi(\mathbf{s})|^2$. Proposal: pick a site k uniformly, propose $\mathbf{s} \to \mathbf{s}^{(k)}$. The proposal is symmetric, so the Metropolis–Hastings acceptance reduces to

$$A(\mathbf{s} \to \mathbf{s}^{(k)}) \;=\; \min\left(1,\; \left|\frac{\psi(\mathbf{s}^{(k)})}{\psi(\mathbf{s})}\right|^2\right) \;=\; \min\Big(1,\; \exp\left[2\,\mathrm{Re}\;\Delta\log\psi\right]\Big), \tag{5.4}$$

with $\Delta\log\psi$ from (5.3). Note the factor of 2 and the Re: dropping either gives a chain that samples $|\psi|$ or ψ rather than $|\psi|^2$, converges to *something*, and is wrong in a way that is invisible without a reference.

Practical parameters

- **Sweep** = N single-flip proposals. Record a sample once per sweep, not once per flip; consecutive flips are almost perfectly correlated.
- **Burn-in** of $\sim$ 8–16 sweeps discarded per iteration.
- **Multiple chains** advanced in lockstep (vectorised over a leading batch axis) rather than one long chain. Sixty-four chains is plenty at these sizes, and it gives you a free convergence diagnostic — the spread of per-chain means.
- **Acceptance rate** should land in 0.3–0.7. Near 0 the state is too sharply peaked for single flips; near 1 the state is nearly uniform (typical at initialisation, and not by itself a problem).

Ergodicity. Single spin flips connect all of configuration space for the TFI chain, so this sampler is ergodic. For a Heisenberg model at fixed total S^z it is *not* — flips change the magnetisation sector — and you must use exchange (swap) moves instead. If you extend this project to Heisenberg, changing the sampler is not optional.

```python
def sample(rbm, n_chains, n_sweeps, n_discard, rng):
    N = rbm.N
    s = rng.choice([-1.0, 1.0], size=(n_chains, N))
    th = rbm.theta(s)                              # cached angles
    out_s, accepted, total = [], 0, 0
    for sweep in range(n_sweeps + n_discard):
        for _ in range(N):
            k   = rng.integers(0, N, size=n_chains)
            idx = np.arange(n_chains)
            sk  = s[idx, k]
            th_new = th - 2.0 * sk[:, None] * rbm.W[k, :]     # Eq. (theta_update)
            dlog   = (-2.0 * sk * rbm.a[k]
                      + np.sum(logcosh(th_new) - logcosh(th), axis=1))
            acc = rng.random(n_chains) < np.exp(2.0 * np.real(dlog))
            s[idx[acc], k[acc]] *= -1.0
            th[acc] = th_new[acc]
            accepted += acc.sum(); total += n_chains
        if sweep ≥ n_discard:
            out_s.append(s.copy())
    return np.concatenate(out_s, axis=0), accepted / total
```

Listing 5.2: Vectorised Metropolis sampler with cached angles. `n_chains` chains advance in lockstep; one configuration per chain is recorded per sweep.

5.6 The local energy

Equation (2.5) translates directly, reusing the same ratio machinery. The diagonal part is a bond sum; the off-diagonal part is N applications of (5.3), one per site, each vectorised over the whole sample batch.

```python
def local_energy(rbm, s, J=1.0, h=1.0, pbc=True):
    ns, N = s.shape
    bonds = N if pbc else N - 1
    diag = -J * sum(s[:, i] * s[:, (i + 1) % N] for i in range(bonds))
    th = rbm.theta(s)
    off = np.zeros(ns, dtype=complex)
    for k in range(N):                                  # N single-flip terms
        th_new = th - 2.0 * s[:, [k]] * rbm.W[k, :]
        dlog = (-2.0 * s[:, k] * rbm.a[k]
                + np.sum(logcosh(th_new) - logcosh(th), axis=1))
        off += np.exp(dlog)
    return diag - h * off
```

Listing 5.3: Local energy for the TFI chain, batched over samples.

The exponential in the last line is the only place amplitudes are materialised, and it is safe precisely because $\Delta\log\psi$ is a *difference*, which stays $O(1)$ for a sensible state. If it overflows, that is diagnostic information: the network has developed a configuration whose amplitude is astronomically larger than its single-flip neighbours, which means the optimisation has diverged. Do not paper over it with a clip; find out why.

5.7 Log-derivatives, batched

From (3.6), with $t_j = \tanh\theta_j$:

```python
def log_derivs(self, s):
    """O_k(s) = d log psi / d p_k. Returns (n_samples, n_par)."""
    t   = np.tanh(self.theta(s))          # (ns, M)
    O_a = s                               # (ns, N)
    O_b = t                               # (ns, M)
    O_W = s[:, :, None] * t[:, None, :]   # (ns, N, M)
    return np.concatenate([O_a, O_b, O_W.reshape(len(s), -1)], axis=1)
```

Listing 5.4: Closed-form log-derivatives. Note $O_{W_{ij}} = s_i t_j$ is an outer product, formed by broadcasting.

The parameter packing order — $[\mathbf{a}, \mathbf{b}, \mathrm{vec}(W)]$ — must match between `log_derivs`, `get_params` and `set_params`. A mismatched `reshape` here produces an optimiser that still decreases the energy (because $S^{-1}F$ is still a descent-ish direction) but converges to the wrong place. Test it: perturb one packed entry, confirm the intended weight moved.

5.8 Assembling and solving the SR equations

```python
def sr_step(rbm, s, eloc, lr, lam):
    ns = len(s)
    O  = rbm.log_derivs(s)
    dO = O - O.mean(axis=0)               # centring: pure variance reduction
    de = eloc - eloc.mean()
    F  = dO.conj().T @ de / ns            # force,  Eq. (force)
    S  = dO.conj().T @ dO / ns            # QGT,    Eq. (qgt)
    S  = S + lam * np.eye(len(F)) * np.maximum(np.real(np.diag(S)), 1.0)
    dtheta = np.linalg.solve(S, F)
    rbm.set_params(rbm.get_params() - lr * dtheta)
```

Listing 5.5: One stochastic-reconfiguration step, with diagonal-shift regularisation.

Three notes. (i) The `np.maximum(..., 1.0)` floor on the diagonal shift prevents the regulariser from vanishing for parameters that happen to have tiny S_{kk} — exactly the gauge directions of §3.5. (ii) `np.linalg.solve` is fine at $|\mathcal{W}| \sim 10^3$; beyond that use `scipy.sparse.linalg.cg` with a matrix-free operator $v \mapsto \bar{O}^\dagger(\bar{O}v)/N_s + \lambda v$, which never forms S. (iii) Dropping SR entirely is a two-line change and is worth keeping as an option for comparison:

```python
def sgd_step(rbm, s, eloc, lr):
    O  = rbm.log_derivs(s)
    dO = O - O.mean(axis=0)
    F  = dO.conj().T @ (eloc - eloc.mean()) / len(s)
    rbm.set_params(rbm.get_params() - lr * F)
```

Listing 5.6: Plain steepest descent, for comparison (Table 4.1).

5.9 The network class, complete

```python
class RBM:
    """log psi(s) = a . s + sum_j log(2 cosh(theta_j)),  theta = b + W^T s."""

    def __init__(self, N, alpha, sigma=0.01, seed=0, complex_params=True):
        rng = np.random.default_rng(seed)
        self.N, self.M = N, alpha * N
        def draw(shape):
            x = rng.normal(0, sigma, shape)
            if complex_params:
```

```
                x = x + 1j * rng.normal(0, sigma, shape)
            return x
        self.a = draw(N); self.b = draw(self.M); self.W = draw((N, self.M))

    @property
    def n_par(self):
        return self.N + self.M + self.N * self.M

    def get_params(self):
        return np.concatenate([self.a, self.b, self.W.ravel()])

    def set_params(self, p):
        N, M = self.N, self.M
        self.a = p[:N].copy()
        self.b = p[N:N + M].copy()
        self.W = p[N + M:].reshape(N, M).copy()

    def theta(self, s):                    # (..., N) -> (..., M)
        return self.b + s @ self.W

    def log_psi(self, s):
        return s @ self.a + np.sum(logcosh(self.theta(s)), axis=-1) \
               + self.M * np.log(2.0)
```

Listing 5.7: The RBM. Parameters are complex by default; `complex_params=False` gives the real ansatz, which is sufficient for the stoquastic TFI chain and roughly twice as fast.

Initialisation

Draw $\mathcal{W} \sim \mathcal{N}(0, \sigma^2)$ with $\sigma \approx 0.01$. Small σ makes ψ nearly uniform, which gives an ergodic, high-acceptance chain and a well-conditioned S; it is the analogue of small-weight initialisation in ordinary deep learning and for the same reason. Large σ produces a sharply peaked initial state, a sampler that sticks, and an S estimated from a handful of effectively distinct configurations.

5.10 The PyTorch port

> **Not executed in this environment.** PyTorch was unavailable when this document was compiled, so unlike everything else here the listings in this section have not been run. The mathematics is derived below and the NumPy equivalents *are* validated; treat the code as a specification to be tested, not as verified output.

Two things change when moving to PyTorch, and both are subtle.

5.10.1 Complex parameters and the Wirtinger convention

PyTorch supports `torch.complex128` parameters and autograd through them. The convention is that for a real-valued scalar loss L and complex parameter p, `p.grad` holds the *conjugate* Wirtinger derivative $\partial L/\partial p^*$. This is chosen precisely so that `p -= lr * p.grad` is steepest descent on L. It is the right convention, but it means a hand-derived gradient must be compared against `p.grad` with the conjugation convention in mind, not blindly.

5.10.2 The surrogate objective

Since autograd cannot pass through the sampler, we need an objective whose gradient *equals* (2.7) while being an ordinary differentiable function of the parameters. Take

$$\mathcal{L} \;=\; \frac{2}{N_s}\,\mathrm{Re}\sum_{n=1}^{N_s}\big[\varepsilon_n^{*}\,\log\psi(\mathbf{s}_n)\big], \qquad \varepsilon_n = \mathrm{stopgrad}\big[E_{\mathrm{loc}}(\mathbf{s}_n) - \bar{E}\big]. \tag{5.5}$$

Proposition 5.2. *$\partial\mathcal{L}/\partial p_k^* = F_k$, the SR force (2.8); and for real parameters $\partial\mathcal{L}/\partial p_k = \partial E/\partial p_k$ as given by (2.7).*

Proof. Write $\mathcal{L} = \frac{1}{N_s}\sum_n[\varepsilon_n^* \log\psi_n + \varepsilon_n\overline{\log\psi_n}]$. The samples and ε are detached, and $\log\psi$ is holomorphic in p, so $\partial(\log\psi_n)/\partial p_k^* = 0$ while $\partial\overline{\log\psi_n}/\partial p_k^* = \overline{O_k(\mathbf{s}_n)}$. Hence $\partial\mathcal{L}/\partial p_k^* = \frac{1}{N_s}\sum_n \varepsilon_n O_k^*(\mathbf{s}_n) = \big\langle (E_{\mathrm{loc}} - \bar{E})\, O_k^* \big\rangle = F_k$. Taking twice the real part recovers (2.7). □

The centring of ε by $\bar{E}$ is what makes the estimator low-variance; it is not optional in practice even though it is exactly zero in expectation.

```python
import torch

def vmc_step(rbm, sampler, optimizer, hamiltonian):
    with torch.no_grad():
        s = sampler.sample(rbm)                       # (ns, N), no grad path
        eloc = hamiltonian.local_energy(rbm, s)       # (ns,) complex
    ebar = eloc.mean()
    eps  = (eloc - ebar).detach()

    logpsi = rbm.log_psi(s)                           # (ns,) complex, WITH grad
    loss   = 2.0 * torch.real(torch.conj(eps) * logpsi).mean()

    optimizer.zero_grad(set_to_none=True)
    loss.backward()                                   # grad = Eq. (grad)
    optimizer.step()
    return ebar.real.item(), eloc.var().real.item()
```

Listing 5.8: PyTorch training step. *Specification only — untested here.* Note `detach()` on the local energies and the absence of any gradient path through the sampler.

To add SR on top of this, replace `optimizer.step()` with an explicit solve: build the Jacobian $\bar{O}$ with `torch.func.jacrev` or `vmap`-ed `grad` over samples, then apply the minSR form of §4.2. `torch.func` is the right tool here; looping `backward()` per sample is N_s times slower and will dominate the runtime.

5.11 Exact-summation mode: the debugging superpower

At $N \le 16$ the entire Hilbert space fits in memory, and you should exploit that ruthlessly. Enumerate all 2^N configurations, evaluate $\log\psi$ on all of them, and compute E by *exact summation* with no sampling at all:

$$E_{\mathrm{exact}}[\mathcal{W}] \;=\; \sum_{\mathbf{s}} p(\mathbf{s})\, E_{\mathrm{loc}}(\mathbf{s}), \qquad p(\mathbf{s}) = \frac{|\psi(\mathbf{s})|^2}{\sum_{\mathbf{s}'}|\psi(\mathbf{s}')|^2}. \tag{5.6}$$

```python
def all_configs(N):
    k = np.arange(1 << N)
    bits = ((k[:, None] >> np.arange(N)) & 1)
    return 1.0 - 2.0 * bits                       # (2**N, N) of +/-1

def exact_energy(rbm, N, J=1.0, h=1.0, pbc=True):
    s  = all_configs(N)
    lp = rbm.log_psi(s)
    lp = lp - np.max(np.real(lp))                 # log-sum-exp stabilisation
    p  = np.abs(np.exp(lp)) ** 2
    p  = p / p.sum()
    return np.sum(p * local_energy(rbm, s, J, h, pbc)).real
```

Listing 5.9: Full-basis enumeration and exact energy. Everything is stabilised by subtracting the maximum log-amplitude before exponentiating.

This gives you three things no amount of staring at convergence plots will:

1. **Separation of error sources.** If exact-summation optimisation reaches E_0 but sampled optimisation does not, the bug is in the sampler. If neither does, the bug is in the ansatz, the local energy, or the gradient.
2. **A noise-free final number.** All energies quoted in this document's tables are exact-summation evaluations of the converged network, so they carry zero Monte Carlo error and the quoted relative errors are purely variational.
3. **A finite-difference gradient check.** Perturb $p_k \to p_k \pm \epsilon$, recompute E_{exact}, compare $[E_+ - E_-]/2\epsilon$ against (2.7). Agreement to $\sim 10^{-6}$ relative confirms the gradient; disagreement by a factor of exactly 2, or by a sign, or by a complex conjugate, tells you which convention you got wrong.

Recommended development order. (1) ED reference energies. (2) $\log\psi$ against a brute-force evaluation of the hidden-unit sum (3.1) at $M \leq 4$. (3) E_{loc} against $\langle \mathbf{s}| \hat{H} |\Psi\rangle / \psi(\mathbf{s})$ computed from the dense $\hat{H}$. (4) Gradient against finite differences. (5) Optimise by exact summation until you hit E_0. (6) *Only then* switch on Monte Carlo. Skipping to (6) is how this project consumes three weeks.

5.12 Pitfalls

Symptom	Likely cause	Check
Energy converges below E_0	Local energy or ED reference wrong; PBC/OBC mismatch	The variational bound cannot be violated. Compare bond counts.
Energy plateaus 10^{-3} above E_0	No SR; or λ too large	Table 4.1. Try exact-summation optimisation.
`nan` after a few iterations	Overflow in $\exp(\Delta \log\psi)$; learning rate too high	Check $\log\cosh$ stability; reduce η; increase λ.
Acceptance rate ≈ 0	Sharply peaked ψ; or Re/factor-2 error in acceptance	Print acceptance every iteration, always.
Sampled E disagrees with exact-summation E	Sampler not at stationarity; too few sweeps; stale θ cache	Assert cached θ equals recomputed θ.
Gradient off by factor 2	Real-vs-complex convention in (2.7)	Finite-difference check.
Optimiser "works" but slowly	Parameter packing mismatch between O_k and `set_params`	Perturb one packed entry, confirm intent.
Results irreproducible	Unseeded RNG	Seed everything; log the seed.

5.13 Testing checklist

1. $\log\cosh$: agrees with `log(cosh(x))` for $|x| < 10$, finite for $x = 10^3$, correct for complex x against `mpmath`.
2. $\log\psi$: matches the explicit 2^M-term sum (3.1) for $M \leq 4$.

3. Ratio identity (5.3): matches $\log \psi(\mathbf{s}^{(k)}) - \log \psi(\mathbf{s})$ computed independently, for random $\mathbf{s}$, k.
4. Sampler: with $\mathcal{W}$ frozen, the empirical histogram over 2^N configurations converges to $p(\mathbf{s})$ from `exact_energy`'s normalisation. A χ^2 or total-variation check.
5. E_{loc}: for a state you can write down (e.g. the exact ψ_0 from ED, fed in by overriding `log_psi`), $E_{\text{loc}}(\mathbf{s})$ is constant $= E_0$ for every $\mathbf{s}$ — the zero-variance property, and the sharpest single test of the operator code.
6. Gradient: finite differences, §5.11.
7. S: Hermitian, positive semi-definite, with at least the 2^M gauge zero modes of §3.5.
8. End-to-end: $N = 6$, $\alpha = 2$ reaches $|E - E_0|/|E_0| < 10^{-6}$.

Test 5 is the one people skip and the one that catches the most. It costs six lines.

5.14 Performance

At fixed α the cost per iteration is dominated by three terms:

Stage	Cost	Comment
Sampling	$O(n_{\text{ch}} \cdot n_{\text{sw}} \cdot N \cdot M)$	With cached angles. Without, an extra factor N.
Local energy	$O(N_s \cdot N \cdot M)$	N flip terms, each $O(M)$ per sample.
O matrix	$O(N_s \cdot NM)$ time, $O(N_s NM)$ memory	Usually the memory bound.
SR solve	$O(N_s \lvert\mathcal{W}\rvert^2 + \lvert\mathcal{W}\rvert^3)$	Replace with minSR / CG when $\lvert\mathcal{W}\rvert \gtrsim 10^3$.

For calibration: the runs in this document ($N \leq 12$, $\alpha = 2$, 64 chains, 16 sweeps, 350 iterations) take 10–40 seconds each in pure NumPy on one core. There is no need for a GPU anywhere in the base project; save that for the extensions.

CHAPTER 6

The Exact Benchmark

6.1 Sparse construction and diagonalisation

Index basis states by integers $k \in [0, 2^N)$, with bit i of k encoding $s_i = 1 - 2b_i$. Then $\hat{\sigma}^x_i$ is the bit flip $k \mapsto k \oplus 2^i$, and the Hamiltonian is assembled in one pass:

```python
from scipy.sparse import csr_matrix
from scipy.sparse.linalg import eigsh

def tfi_sparse(N, J=1.0, h=1.0, pbc=True):
    dim, rows, cols, vals = 1 << N, [], [], []
    bonds = N if pbc else N - 1
    for k in range(dim):
        s = np.array([1 - 2 * ((k >> i) & 1) for i in range(N)])
        diag = -J * sum(s[i] * s[(i + 1) % N] for i in range(bonds))
        rows.append(k); cols.append(k); vals.append(diag)
        for i in range(N):                      # off-diagonal: single flips
            rows.append(k ^ (1 << i)); cols.append(k); vals.append(-h)
    return csr_matrix((vals, (rows, cols)), shape=(dim, dim))

def ed_ground(N, J=1.0, h=1.0, pbc=True):
    H = tfi_sparse(N, J, h, pbc)
    if (1 << N) < 200:
        w, v = np.linalg.eigh(H.toarray()); return w[0], v[:, 0]
    w, v = eigsh(H, k=1, which="SA");          return w[0], v[:, 0]
```

Listing 6.1: Sparse TFI Hamiltonian and ground state. Exact by construction; this is the reference every variational number is measured against.

Dense `eigh` is fine to $N \approx 14$; Lanczos (`eigsh`) with `which="SA"` reaches $N \approx 24$ on a laptop and $N \approx 30$ with symmetry sectors and patience. Beyond that, DMRG in 1D.

Table 6.1: Exact ground-state energies, TFI chain, PBC, $J = h = 1$ (the critical point). Computed with the code above.

N	Hilbert dim	E_0	E_0/N
6	64	-7.72740661	-1.28790110
8	256	-10.25166179	-1.28145772
10	1024	-12.78490644	-1.27849064
12	4096	-15.32259515	-1.27688293
∞	—	—	$-4/\pi = -1.27323954$

6.2 The free-fermion solution

The 1D TFI chain is exactly solvable, a fact the project brief does not mention and which gives you a benchmark at *any* N, not just where exact diagonalisation reaches. A Jordan–Wigner transformation maps (1.2) to free fermions, and a Bogoliubov rotation diagonalises the result:

$$\hat{H} = \sum_{k} \varepsilon_k \left(\eta_k^\dagger \eta_k - \tfrac{1}{2} \right), \qquad \varepsilon_k = 2\sqrt{J^2 + h^2 - 2Jh\cos k}, \tag{6.1}$$

so that

$$E_0 = -\tfrac{1}{2}\sum_k \varepsilon_k = -\sum_k \sqrt{J^2 + h^2 - 2Jh\cos k}. \tag{6.2}$$

In the thermodynamic limit this becomes

$$\frac{E_0}{N} = -\frac{1}{2\pi}\int_0^{2\pi} \sqrt{J^2 + h^2 - 2Jh\cos k}\; dk \;\xrightarrow[J=h=1]{}\; -\frac{4}{\pi} = -1.27323954\ldots, \tag{6.3}$$

consistent with the last column of Table 6.1.

The parity subtlety. At finite N with periodic boundary conditions, the Jordan–Wigner string makes the fermion problem *antiperiodic* in the even-parity sector and periodic in the odd one, so the allowed momenta are $k_n = (2n+1)\pi/N$ in one sector and $2\pi n/N$ in the other, and the true ground state is the lower of the two sector minima. Getting this wrong produces a "free-fermion benchmark" that disagrees with exact diagonalisation at the third decimal and sends you hunting for a bug in the NQS code. For $N \le 12$, just use exact diagonalisation (Table 6.1); reserve the analytic formula for the $N > 20$ extension, and validate it against ED in the overlap region first.

CHAPTER 7

Protocol, Diagnostics and Deliverables

7.1 Results the project asks for

Table 7.1 is the deliverable "ML versus exact E_0 across system size N", produced by the reference implementation. Every variational number is an exact-summation evaluation of the converged network: no Monte Carlo error is folded into the comparison.

Table 7.1: Reference results. TFI chain, PBC, $J = h = 1$, RBM with $\alpha = 2$ (complex parameters), 350 SR iterations, 64 chains, 16 sweeps per iteration, $\eta = 0.05$, $\lambda_t = \max(10^{-2} \cdot 0.99^t, 10^{-5})$, seed 7.

N	Hilbert dim	$\lvert\mathcal{W}\rvert$	E_0 (exact)	E_{NQS}	rel. error
6	64	90	-7.72740661	-7.72740619	5.4×10^{-8}
8	256	152	-10.25166179	-10.25164020	2.1×10^{-6}
10	1024	230	-12.78490644	-12.78482480	6.4×10^{-6}
12	4096	324	-15.32259515	-15.32243025	1.1×10^{-5}

Table 7.2: Expressivity: relative error against hidden density at $N = 10$, 350 SR iterations, seed 5. Monotone improvement with α, as the nesting of the ansatz family requires.

α	M	$\lvert\mathcal{W}\rvert$	rel. error
1	10	120	3.6×10^{-5}
2	20	230	6.3×10^{-6}
4	40	450	2.8×10^{-6}

Reading Table 7.1. The relative error grows with N at fixed α and fixed iteration count. This is *not* a failure of the ansatz — it is mostly optimisation, not expressivity: the parameter count grows as N^2 while the sample budget and iteration count are held constant. Disentangling the two requires holding the *converged* error fixed and reporting the α or the iteration count needed to reach it. Reporting the raw error against N without that caveat invites a false conclusion about scaling.

7.2 Diagnostics to log every run

- $\langle E_{\mathrm{loc}} \rangle$ and its error bar with autocorrelation correction.
- $\mathrm{Var}[E_{\mathrm{loc}}]$ — the zero-variance diagnostic, and the one that survives past ED.
- Metropolis acceptance rate.
- $\|\delta\mathcal{W}\|$ per step, and the condition number or smallest retained eigenvalue of the regularised S.

- Spread of per-chain energy means (a cheap $\hat{R}$-like convergence check).
- The seed, the git commit, and the full parameter dictionary — the brief's instruction to keep a dated log of every run is not bureaucracy; without it you cannot tell a hyperparameter effect from a code change.

7.3 Suggested timeline, revised

The brief allocates two weeks to tooling, two to exact diagonalisation, two to the NQS. The exact-diagonalisation code above is an afternoon, not two weeks. A more realistic split:

Week	Focus	Exit criterion
1	Tooling + ED	Table 6.1 reproduced; PyTorch autograd understood on a toy problem.
2	Ansatz + estimators	$\log\psi$, E_{loc}, gradient all pass the tests of §5.13.
3	Exact-summation optimisation	$N = 6, 8$ reach E_0 to 10^{-6} with no sampling.
4	Monte Carlo	Sampled and exact-summation energies agree within error bars.
5	SR + production runs	Tables 7.1 and 7.2; convergence plots.
6	Extension + writeup	One item from Chapter 8.

CHAPTER 8

Extensions Toward Research

8.1 Past exact diagonalisation

Push N to 40–100 with PBC. The 1D TFI remains free-fermion solvable, so you keep an exact benchmark the whole way — a rare and valuable situation. The interesting question becomes how α must scale with N to hold the relative error fixed. Watch for: sampling cost growing, $|\mathcal{W}| \sim \alpha N^2$ making the explicit SR solve untenable (switch to minSR), and autocorrelation time lengthening near criticality.

8.2 Symmetry

The TFI chain with PBC is translation invariant and has a $\mathbb{Z}_2$ spin-flip symmetry ($\prod_i \hat{\sigma}_i^x$). Imposing translation invariance by weight sharing — W_{ij} depending only on $i-j$ mod N, giving α *filters* rather than M independent hidden units — cuts the parameter count by a factor N and typically *improves* accuracy, because it projects onto the correct momentum sector. This is the cheapest available accuracy gain and the natural first extension.

8.3 Where the method earns its keep: sign structure

Move to the frustrated J_1–J_2 chain or the 2D square-lattice model. The ground state is no longer positive, complex parameters become load-bearing, and the optimisation becomes genuinely hard — the amplitude and the sign structure must be learned simultaneously and the latter is far less smooth. Standard techniques: apply the Marshall sign rule as a fixed prefactor and learn the residual; use deeper or convolutional architectures; use many more samples. This is an open research area, not a solved one, and it is where "expressivity affects accuracy" stops being a textbook exercise.

8.4 Architecture

Swap the RBM for a CNN or an autoregressive model (§3.6) and measure accuracy at matched parameter count. Autoregressive models are especially attractive here because exact sampling removes autocorrelation entirely — which means an architecture comparison is no longer confounded by how well each ansatz mixes under Metropolis.

8.5 Dynamics

The second half of the Carleo–Troyer paper, which the project brief omits, uses the same machinery for real-time evolution via the time-dependent variational principle: replace the imaginary-time step of §4.2 with a real one, $S\,\dot{\mathcal{W}} = -iF$. The estimators are identical; only the update changes. A quench of the transverse field at $N \leq 12$ can be benchmarked directly against exact $e^{-i\hat{H}t}$ propagation, which makes it a clean and self-contained sixth-week project.

APPENDIX A

Notation

Symbol	Type	Meaning
N, M	int	Visible spins; hidden units. $M = \alpha N$.
$\mathbf{s}$	$\{\pm 1\}^N$	Spin configuration; a basis state of $\mathcal{H}$.
$\mathbf{s}^{(k)}$	$\{\pm 1\}^N$	$\mathbf{s}$ with spin k flipped.
$\psi(\mathbf{s})$	$\mathbb{C}$	Unnormalised amplitude.
$\mathcal{W} = \{a_i, b_j, W_{ij}\}$	$\mathbb{C}^{\lvert\mathcal{W}\rvert}$	Variational parameters; p_k is a generic component.
$\theta_j(\mathbf{s})$	$\mathbb{C}$	Effective angle $b_j + \sum_i W_{ij} s_i$.
$E_{\mathrm{loc}}(\mathbf{s})$	$\mathbb{C}$	Local energy, Eq. (2.3).
$O_k(\mathbf{s})$	$\mathbb{C}$	Log-derivative $\partial_{p_k} \log \psi$.
F_k	$\mathbb{C}$	Force, Eq. (2.8).
$S_{kk'}$	$\mathbb{C}$	Quantum geometric tensor, Eq. (4.1).
η, λ	$\mathbb{R}$	Learning rate; SR regularisation.
N_s, τ	int, $\mathbb{R}$	Sample count; integrated autocorrelation time.

APPENDIX B

The One-Page Algorithm

Variational Monte Carlo with stochastic reconfiguration.

1. Initialise $\mathcal{W} \sim \mathcal{N}(0, \sigma^2)$, $\sigma \approx 0.01$.
2. **repeat** for $t = 1, \ldots, T$:
   a. Sample $\{\mathbf{s}_n\}_{n=1}^{N_s} \sim |\psi_{\mathcal{W}}|^2$ by Metropolis single flips, using cached θ and the ratio identity (5.3); discard burn-in.
   b. Evaluate $E_{\mathrm{loc}}(\mathbf{s}_n)$ by (2.5) — N single-flip ratios plus a bond sum.
   c. Evaluate $O_k(\mathbf{s}_n)$ in closed form, (3.6).
   d. Centre: $\varepsilon_n = E_{\mathrm{loc}}(\mathbf{s}_n) - \bar{E}$, $\bar{O}_{nk} = O_k(\mathbf{s}_n) - \langle O_k \rangle$.
   e. Form $F = \bar{O}^\dagger \varepsilon / N_s$ and $S = \bar{O}^\dagger \bar{O} / N_s$; regularise $S \to S + \lambda_t \,\mathrm{diag}(S)$.
   f. Solve $S\,\delta\mathcal{W} = F$ and set $\mathcal{W} \to \mathcal{W} - \eta\,\delta\mathcal{W}$.
   g. Log $\bar{E}$, $\mathrm{Var}[E_{\mathrm{loc}}]$, acceptance, $\|\delta\mathcal{W}\|$.
3. Report $E_{\mathrm{exact}}[\mathcal{W}]$ by full summation (if $N \leq 16$) against E_0 from ED.

APPENDIX C

Sources

Founding paper. G. Carleo and M. Troyer, *Solving the quantum many-body problem with artificial neural networks*, Science **355**, 602 (2017), doi:10.1126/science.aag2302; preprint arXiv:1606.02318.

Software. F. Vicentini *et al.*, *NetKet 3: Machine Learning Toolbox for Many-Body Quantum Systems*, SciPost Phys. Codebases **7** (2022), doi:10.21468/SciPostPhysCodeb.7; code at github.com/netket/netket.

Reviews. G. Carleo *et al.*, *Machine learning and the physical sciences*, Rev. Mod. Phys. **91**, 045002 (2019). H. Lange *et al.*, *From architectures to applications: a review of neural quantum states*, Quantum Sci. Technol. **9**, 040501 (2024).

Small-system worked example. H. Saito, *Solving the Bose–Hubbard model with machine learning*, arXiv:1707.09723.

Structural connections. S. R. Clark, *Unifying neural-network quantum states and correlator product states via tensor networks*, arXiv:1710.03545; J. Chen *et al.*, *On the equivalence of restricted Boltzmann machines and tensor network states*, arXiv:1701.04831.

Method. F. Becca and S. Sorella, *Quantum Monte Carlo Approaches for Correlated Systems*, CUP (2017) — the standard reference for stochastic reconfiguration and variational Monte Carlo generally.